

1. Checklist de Teste Estático de Código (Revisão Manual)

Todas as variáveis estão inicializadas antes do uso?

- **Sim.** As variáveis de instância fundamentais (como `treinador1`, `treinador2`, `sc` e `terreno`) são rigorosamente inicializadas através de Injeção de Dependência no construtor da classe `Batalha`. As variáveis de escopo local (como `dano` e `chanceInstakill` no método `atacar`) são inicializadas no exato momento da sua declaração, prevenindo erros de `NullPointerException`.

Há variáveis declaradas e nunca usadas?

- **Não.** A revisão manual e a execução não apontaram variáveis órfãs. Instâncias globais como o `Random random` e o `Scanner sc` são ativamente utilizadas, bem como todas as variáveis instanciadas para a manipulação dos turnos.

Existe código inacessível?

- **Não.** Não foram identificados blocos de código isolados (*dead code*). O uso do comando `return` após a validação do `Instakill` encerra o método precocemente de forma intencional, mas o restante do código permanece acessível para os 98% dos casos em que a probabilidade não se aplica.

Há código duplicado?

- **Não há duplicação crítica.** Ações que ocorrem múltiplas vezes, como a checagem de `status`, foram corretamente isoladas em métodos auxiliares (`aplicarQueimadura`, `aplicarEfeitoTerreno`), evitando a repetição de lógica. A verificação da matriz elemental possui múltiplos `if/else if`, mas trata-se de um fluxo condicional de regras de negócio, não de código duplicado.

Existem erros de sintaxe?

- **Não.** O código-fonte está totalmente em conformidade com as regras de sintaxe da linguagem Java, compilando sem erros (zero marcações vermelhas) na IDE Eclipse e passando pelas validações iniciais do `Checkstyle`.

Há erros de lógica que quebram regras do negócio?

- **Não.** Todas as regras do edital foram preservadas. Foi implementada uma trava estrutural (`if (dano < 1) { dano = 1; }`) que impede o erro lógico fatal de um ataque com dano negativo curar o adversário, e a regra limite de itens foi assegurada com o lançamento de uma `IllegalStateException`.

Há erros de tipagem?

- **Não.** As tipagens primitivas e por referência estão corretas. Destaca-se a aplicação bem-sucedida de *Casting* explícito (`int`) para converter e arredondar com segurança os resultados matemáticos gerados pelas multiplicações decimais (`double`) dos bônus de

terreno e vantagens elementais.

O fluxo de controle é válido (sem loops infinitos, condições impossíveis)?

- **Sim.** O ciclo principal `while (pokesal1.estarVivo() && pokesal2.estarVivo())` possui uma condição de paragem sólida, pois a matemática garante que o HP sofrerá reduções até chegar a zero, evitando *loops* infinitos. Condições impossíveis nos menus de seleção foram tratadas com a cláusula `else`, lançando `IllegalArgumentException` se o utilizador tentar opções inexistentes.

Outros tipos de erros identificados?

- **Apontamento Arquitetural (Code Smell):** Como documentado na Fase 01, a classe `Batalha` mistura a lógica central da aplicação com funções de Entrada/Saída (`System.out.println`), violando parcialmente o Princípio da Responsabilidade Única (SRP). Contudo, isto não é um erro de execução, mas sim uma decisão arquitetural justificada para viabilizar a entrega do MVP funcional via consola nesta fase.